The Relay — Engineering Report

1. What I built and why

I built **The Relay**, a five-stage asynchronous research assistant in Python (FastAPI, FAISS, `sentence-transformers/all-MiniLM-L6-v2`, Groq SDK) answering engineering questions strictly from three arXiv papers: *Attention Is All You Need* (`1706.03762v7`), *Moshi* (`2410.00037v1`), and *Dynamic Chunking / H-Net* (`2507.07955v2`). The pipeline executes: (1) `QuestionAnalyzer`, (2) `LibraryRetriever.search_library` (formal tool call), (3) `DecisionEngine` (confidence gate + 1x query refinement loop), (4) `AnswerGenerator`, and (5) `AnswerVerifier` (citation/page check + 1x refinement loop). **The key shaping decision** was placing a local sliding-window `RateLimiter` (RPM/TPM/RPD/TPD) and bounded FIFO `ConcurrencyAndQueueManager` *before* the model call, paired with `AnswerVerifier` *after* it—refusing or fast-rejecting rather than sending unverified answers or burning quota on doomed requests.

2. The real service and my stand-in

Using `openai/gpt-oss-120b` on Groq's free tier (`max_retries=0`), I ran 5 baseline attempts (`Q01`, `Q04`, `Q07`, `Q13`, `Q02`). **Two actual provider requests succeeded** (`Q01`: `TTFT=1029.33 ms`, `total=1756.97 ms`, `2440/389` tokens; `Q04`: `TTFT=1235.20 ms`, `total=1567.97 ms`, `3438/406` tokens). Because those two RAG prompts consumed **6,673 tokens** in `<6s`, our local `7,000 TPM` guard proactively rejected the remaining **three attempts** (`Q07`, `Q13`, `Q02`) locally (`rate_limited` in `18.95–39.68 ms`) before hitting Groq's `8,000 TPM` ceiling (`x-ratelimit-remaining-tokens` dropped from `5207` to `1808`).

Table 1: Limits found in the real service (`openai/gpt-oss-120b`)

Limit	Value you found	How you found it	Confidence
Requests per minute	`30`	Groq org dashboard (not stressed to 429)	Medium
Tokens per minute	`8,000`	Observed `x-ratelimit-limit-tokens: 8000` header	High
Requests per day	`1,000`	Observed `x-ratelimit-limit-requests: 1000` header	High
Tokens per day	`200,000`	Groq org dashboard	Medium
Requests in flight at once	`not measured`	Not stressed concurrently on real Groq	Low
What it returns when a limit is hit	`not measured` (local guard tripped first)	Local `RateLimiter` blocked at `6,673/7,000` TPM	Low
Time until it recovers	`20.947s`–`46.44s` (tokens), `2m52.8s`–`4m19.2s` (req)	Observed `x-ratelimit-reset-*` headers on `Q01`/`Q04`	High

My FastAPI stand-in (`standin_service.py`, `seed=42`) was calibrated from the `n=2` provider calls: `base_ttft_ms=1132.26`, `ttft_jitter_ms=102.94`, `inter_token_ms=0.72 ms/tok`, with assumed `concurrency_penalty_ms=60.0` and `target_output_tokens=150`. **Where it differs most:** `n=2` cannot capture real tail spikes or non-linear LPU batching saturation, and the stand-in synthesizes deterministic cited text from `[SOURCE: ...]` headers rather than performing 120B reasoning.

3. Results

All figures match `results/results.csv` (`147` rows). For `real_baseline`: across **all 5 attempts**, `total_ms` is `p50=39.7 / p95=1719.2 / p99=1749.4 ms` (fast local rejections pull p50 down); across the **2 successful provider requests** (`Q01`, `Q04`), `TTFT` is `p50=1132.3 / p95=1224.9 ms` (`p99` `not measured`) and `total_ms` is `p50=1662.5 / p95=1747.5 ms`. TTFT is reported only for actual provider requests.

Table 2a: Latency by scenario

Scenario	Real or stand-in	Requests sent	Duration (s)	Offered rate (per min)	First token p50 / p95 / p99 (ms)	Total time p50 / p95 / p99 (ms)
`real_baseline` (all 5 / 2 ok)	real	5 (2 ok, 3 local rl)	13.40	22.4	1132.3 / 1224.9 / not measured *(n=2 ok)*	39.7 / 1719.2 / 1749.4 *(all 5)*
`steady`	stand-in	12	30.36	23.7	2697.8 / 3149.7 / 3172.3	2799.1 / 3465.9 / 3623.7
`burst`	stand-in	18	10.57	102.2	2938.1 / 3162.9 / 3200.5	6481.7 / 8125.9 / 9836.7
`overload`	stand-in	36	8.95	241.3	3110.1 / 3215.2 / 3224.6	33.4 / 3169.2 / 4650.2
`halved_limit`	stand-in	18	17.08	63.2	2578.2 / 2738.2 / 2752.4	39.6 / 2488.8 / 2730.9
`daily_exhausted`	stand-in	10	1.90	315.8	not measured	36.7 / 71.3 / 72.5

Table 2b: Outcomes by scenario

Scenario	Succeeded %	Rate-limited %	Timed out or given up %	Retries per request	Peak queue length	Tokens per answer	Cost per 1,000 answers
`real_baseline`	40.0%	60.0%	0.0%	0.00	0	3336.5	\$0.00 (Free Tier)
`steady`	100.0%	0.0%	0.0%	0.00	0	2347.1	\$0.00 (Free Tier)
`burst`	55.6%	44.4%	0.0%	0.00	10	2330.8	\$0.00 (Free Tier)
`overload`	8.3%	91.7%	0.0%	0.00	6	2481.3	\$0.00 (Free Tier)
`halved_limit`	11.1%	88.9%	0.0%	0.00	0	2098.0	\$0.00 (Free Tier)
`daily_exhausted`	0.0%	100.0%	0.0%	0.00	0	not measured	not measured

Number we could not have guessed without running it: Under Groq's `8,000 TPM` limit, **just 2 RAG requests** (`2,829 + 3,844 = 6,673` tokens) saturated a minute's token budget—capping real throughput at **~2.3 RAG req/min** vs. the `30 RPM` headline limit.

4. Where the time goes

For real Groq request `Q04` (`total_ms = 1567.97 ms` in `results/groq_baseline.json`): **Waiting = `1235.32 ms` (78.8%)** (`queue_wait = 0.12 ms` + pre-token `TTFT = 1235.20 ms` on 3,438 input tokens); **Working = `327.98 ms` (20.9%)** (`question_analysis = 0.50 ms`, `retrieval = 15.51 ms`, `decision = 0.15 ms`, `token_streaming = 311.74 ms` for 406 tokens, `verification = 0.08 ms`); **Retrying = `0.00 ms`**; **Unattributed orchestrator overhead = `4.67 ms` (0.3%)** (`1567.97 - 1563.30` recorded stage sum, covering prompt assembly, rate-limit accounting, and Pydantic serialization). Over 78% of latency occurred **before the first token**.

5. When a limit is hit

At the **per-minute limit**, `RateLimiter` delays requests in the bounded FIFO queue if reset fits within `deadline_seconds` (`peak_queue=10` in `burst`, `total p95=8125.9 ms`), and drops excess requests fast (`33–40 ms` `rate_limited`) when full or past deadline. At the **per-day limit**, `daily_exhausted=True` blocks all external calls immediately. **No one gets an unverified weaker answer.**

Retrieval: quality versus cost

Evaluated 16 questions (`11` answerable, `5` unsupported/trap) in `benchmark_questions.json` using the stand-in:

Table 3: Retrieval settings *(all 16 questions; answerable subset n=11; unsupported/trap subset n=5)*

Setting	Questions asked	Correct and cited %	Wrong %	Correctly refused %	Tokens per answer	First token p50 / p95 (ms)	Total time p50 / p95 (ms)
`S1_250w_k3_norerank`	16	56.2% *(9/16; 81.82% of 11)*	43.8% *(7/16)*	0.0% *(0/5)*	1283.6	2268.5 / 2426.1	2305.6 / 2447.9
`S2_400w_k5_norerank`	16	56.2% *(9/16; 81.82% of 11)*	43.8% *(7/16)*	0.0% *(0/5)*	2482.8	2773.4 / 2948.4	2794.2 / 2964.0
`S3_500w_k7_rerank`	16	62.5% *(10/16; 90.91% of 11)*	37.5% *(6/16)*	0.0% *(0/5)*	4124.9	2595.2 / 2781.8	2618.9 / 2803.6

Critical limitation: The stand-in retrieval evaluation achieved **0/5 (0.0%) correct refusals** on the 5 unsupported/trap questions because domain-adjacent queries scored `0.31–0.60` cosine similarity (passing `DecisionEngine`'s `0.22` floor), and the stand-in synthesizes from `[SOURCE: ...]` headers rather than reading passage bodies—so **real Groq Stage 4/5 refusal behavior is not proven by this stand-in experiment**. **Chosen setting: `S1_250w_k3_norerank`**—it matches `S2` answerable recall (`9/11 = 81.82%`) at **half the token cost (`1283.6` vs `2482.8`)**, doubling throughput under `8,000 TPM`. When close to the TPM limit, we step `top_k` from `5` to `3`. **Proposed (unimplemented) `DecisionEngine` improvements:** add paper-entity mismatch detection (refuse when a named paper lacks matching chunks) and raise/calibrate domain-specific refusal gates on real Groq.

How this library will mislead a naive retriever

1. **Cross-paper entity/metric blending:** *"What BLEU score did Moshi achieve on WMT 2014?"* (`Q12`) — dense search blends *Moshi* with *Attention*'s WMT 2014 BLEU table (`score=0.535`). *Prevention:* Paper-scoped filtering when a single paper is named.

2. **Terminology collision ("chunking"):** *"How does Dynamic Chunking define chunking vs RAG text chunking?"* (`Q11`) — H-Net byte-level routing vs. document splitting. *Prevention:* Hybrid re-ranking (`S3`) and explicit paper ID citations.

3. **Domain-adjacent out-of-scope similarity:** *"How does DeepSeek-V3 implement MLA?"* (`Q14`) — matches *Attention*'s Multi-Head Attention (`score=0.43–0.53`). *Prevention:* Stage 4/5 claim-to-passage verification rather than cosine threshold alone.

6. Retries

`RetryManager` applies exponential backoff with jitter (`0.5–1.5x`), honors `Retry-After`, enforces `max_retries=2` + a `25%` retry budget, and never retries permanent errors (`401`, `400`, `daily_quota_exhausted`), guarded by a 3-state `CircuitBreaker`. **When retrying makes things worse:** Under `8,000 TPM` saturation, retrying 429s with `1–2s` backoff against a `20–46s` token reset window burns retry budget and holds queue slots, starving fresh requests after reset.

7. What the person sees

At 3pm when the daily budget is gone (`daily_exhausted`), a waiting user receives an immediate (`36.7 ms` median) HTTP 200 structured refusal explaining that the daily API budget is exhausted and external calls are paused until daily reset—never hanging or receiving a guessed answer.

8. What I'm unsure about, what I left out, what I learned

- **Least certain assumption:** Linear `concurrency_penalty_ms=60.0 ms` in the stand-in; real Groq may exhibit sharp step-function queueing at high concurrency.

- **Deliberately left out:** (1) A web UI, and (2) running all 48 `retrieval_eval` queries on real Groq (~120k tokens, >50% of daily TPD).

- **What I learned:** On token-capped free tiers, **prompt size (`tokens_in`), not RPM, governs capacity**—reducing `top_k` from 7 to 3 yields more throughput than tuning retry backoff.

Your feedback on Vaani

(Based on live inspection of `https://www.vaani.dev/` on 2026-11-12; ready for your personal review:)

- **What stood out:** Real-time multilingual code-switching (*"Move my visit to कल सुबह दस बजे?"*), ~450ms p90 latency, and "zero dead air" turn-taking examples with Indian enterprise case studies.

- **What was confusing:** Several hero/case-study metric counters render as `0M+`, `0.0%`, `<0ms`, and `Calls attempted 0` in initial HTML before client-side animation triggers.

- **One thing I would change:** Server-render real baseline numbers instead of `0` placeholders and add a 3-line developer SDK quickstart above the fold.

How I worked with the assistant

- **Three prompts that mattered most:** (1) Requiring a full workspace/PDF audit (`PROJECT_STATUS.md`) before Phases 2–7; (2) Enforcing `.env` secret isolation and explicit approval before any real Groq call; (3) Requiring post-rotation security checks and strict numerical/denominator auditing in `REPORT.md`.

- **One thing the assistant got wrong & how I caught it:** `groq_service.py` initially called `raw_resp.parse()` synchronously in `groq==1.7.0`, raising `TypeError` on the first real Groq run (fixed with `await raw_resp.parse()`), and `queue_manager.py` had an async slot-reservation race caught by `test_scenario_8_queue_full` (fixed atomically under `self._lock`).

- **Decisions & ownership:** Selecting `openai/gpt-oss-120b`, providing the Groq Free account & limits (`30 RPM / 8,000 TPM / 1,000 RPD / 200,000 TPD`), rotating the exposed key, and approving the 5-request baseline + single connection test were strictly human-directed.